TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
HCM·UTE
HCMC University of Technology and Engineering

ARIN330585 – Trí tuệ nhân tạo

Introduction to Artificial Intelligence

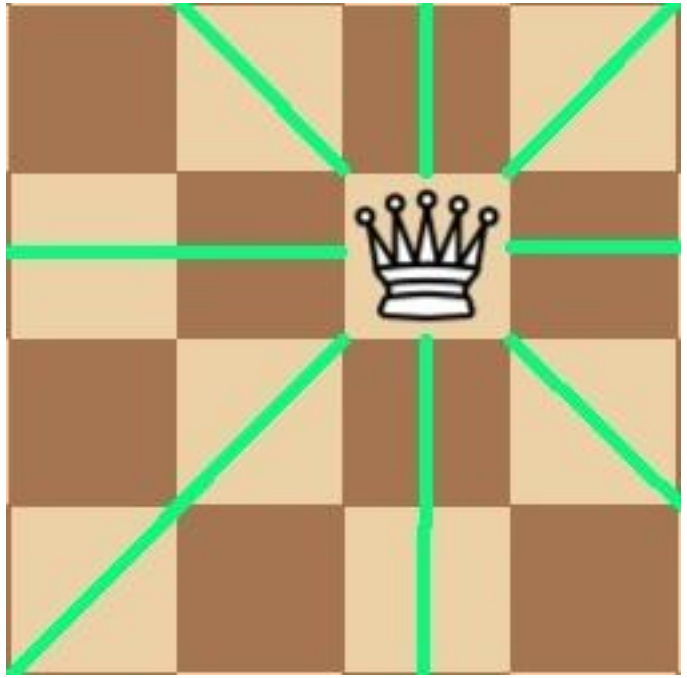
NCS. Võ Long Tuấn

Chapter 6: Local Search

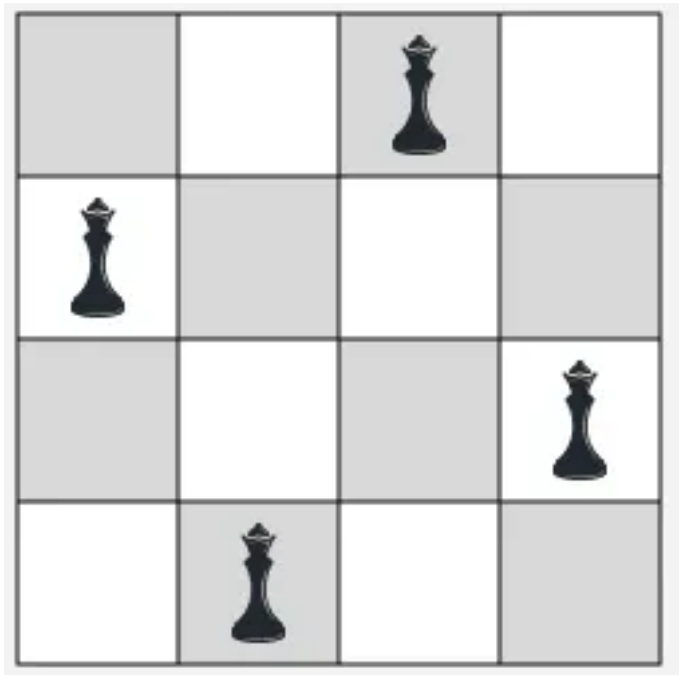
1. *Local search*
2. *Hill climbing*
3. *Simulated annealing*
4. *Genetic algorithms*

1. Local search

4-queens problem



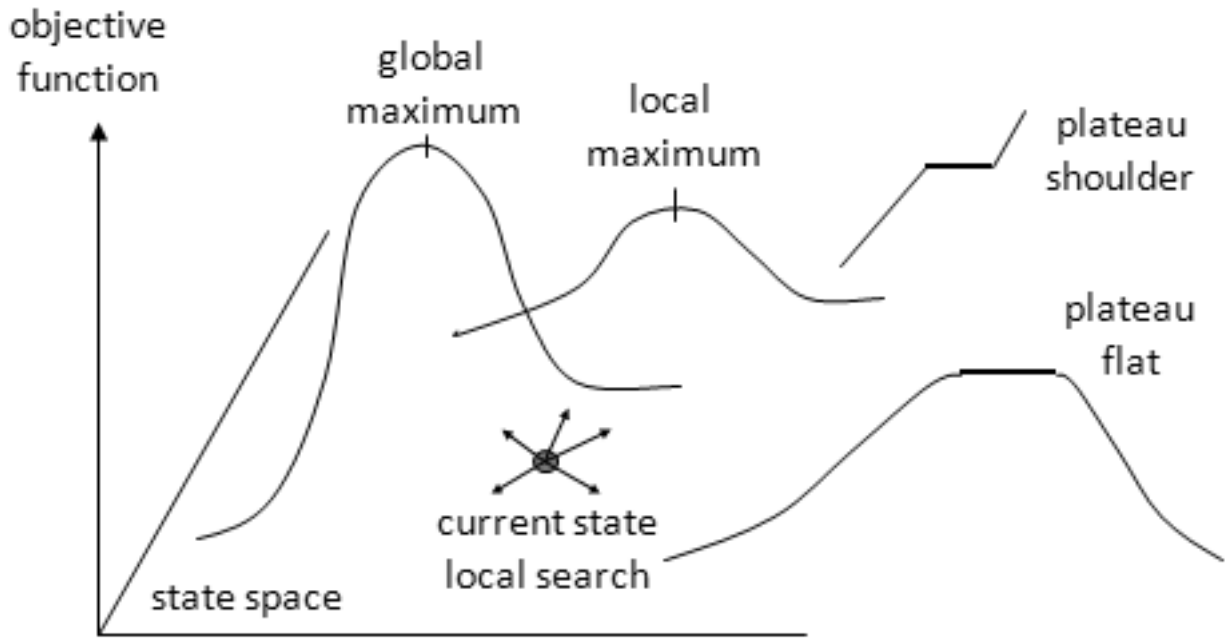
In chess, a queen can move in 8 directions, as shown in the picture.



We try to find the best placement of 4 queens on a 4×4 board such that no two queens attack each other.

Local search

Local search is typically used for combinatorial or discrete optimization problems, where the goal is to find an *optimal state* in a discrete search space, *without caring about the path that leads to it.*

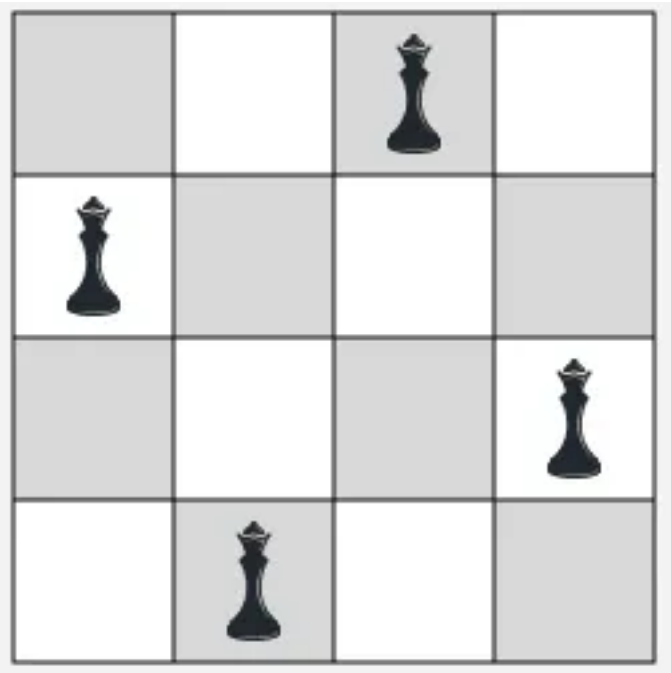


- Local search: iterative improvement.
- New successor function: local changes.
- It's generally much faster and more memory-efficient, but it's incomplete and suboptimal.

Combinatorial optimization problems

Combinatorial or discrete optimization problems:

- Find an optimal state that maximizes or minimizes the objective function.
- Very large state space.
- It is not feasible to use previous search states to explore the entire state space.
- Efficient, also accept approximate solutions.



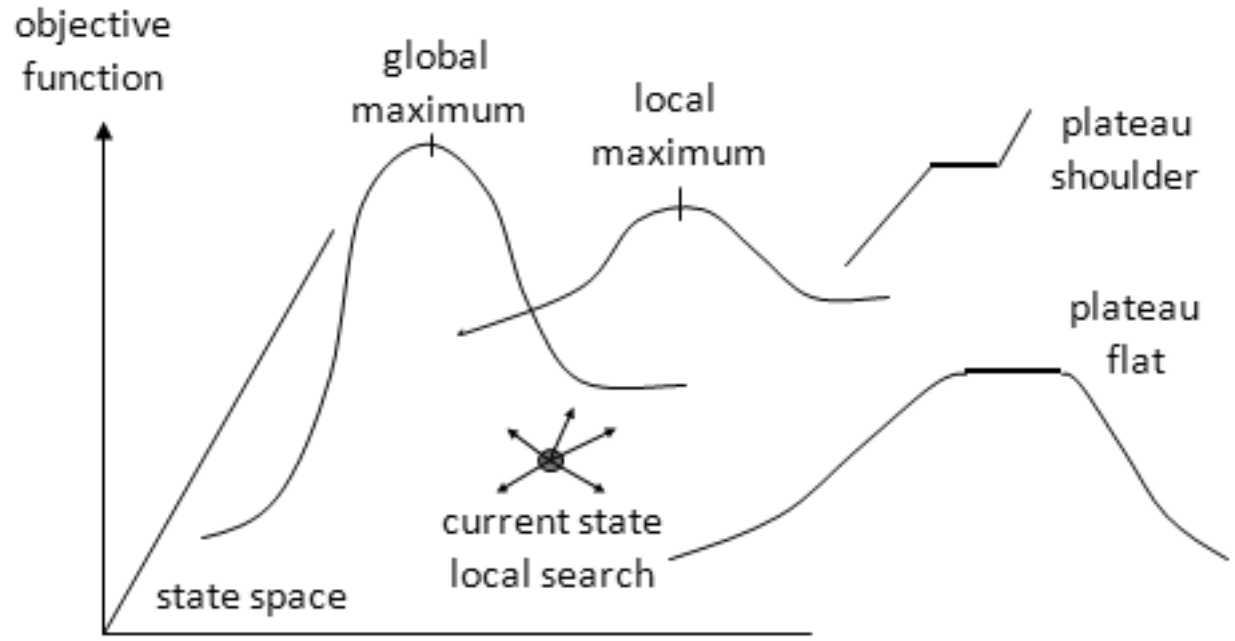
Combinatorial optimization problems

Problem formulation:

- X: State space
- S: Successor
- Obj: Objective function

Find a state X^* such that Obj(X^*) is maximum or minimum.

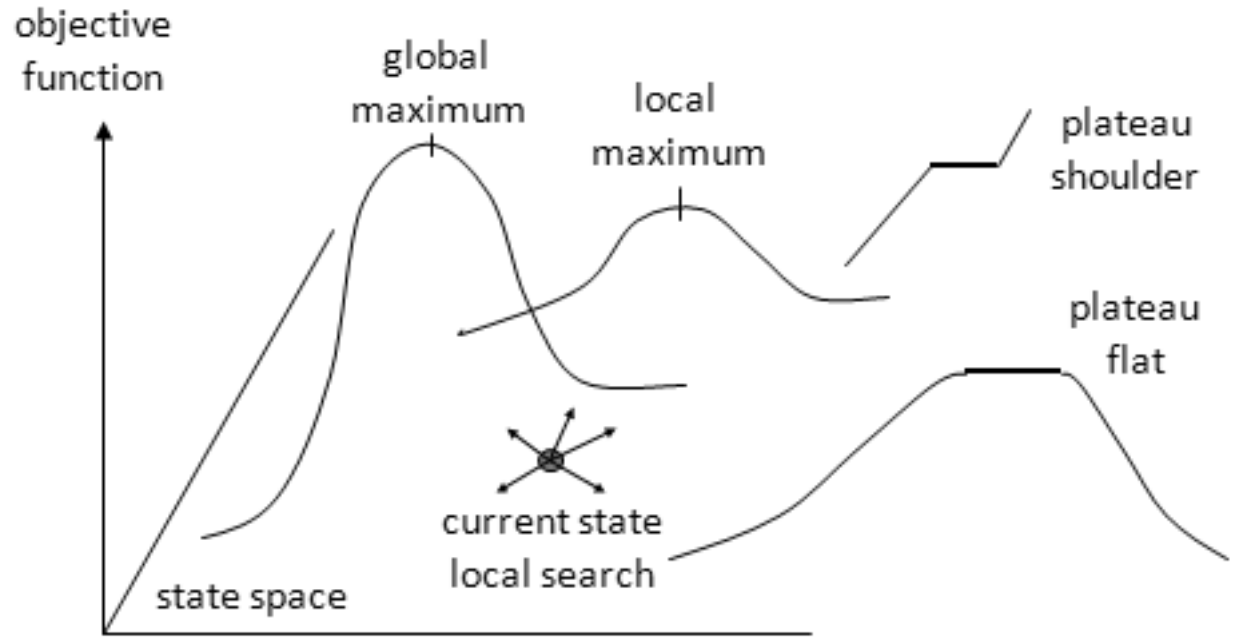
Note that the objective function can be easily transformed by multiplying by -1.



Local search

Idea:

- Focus on the goal, no path needed.
- Consider each state as a solution (non-optimal), and improve the objective by moving to the neighbors of the current state.
- Change a state by performing a successor.

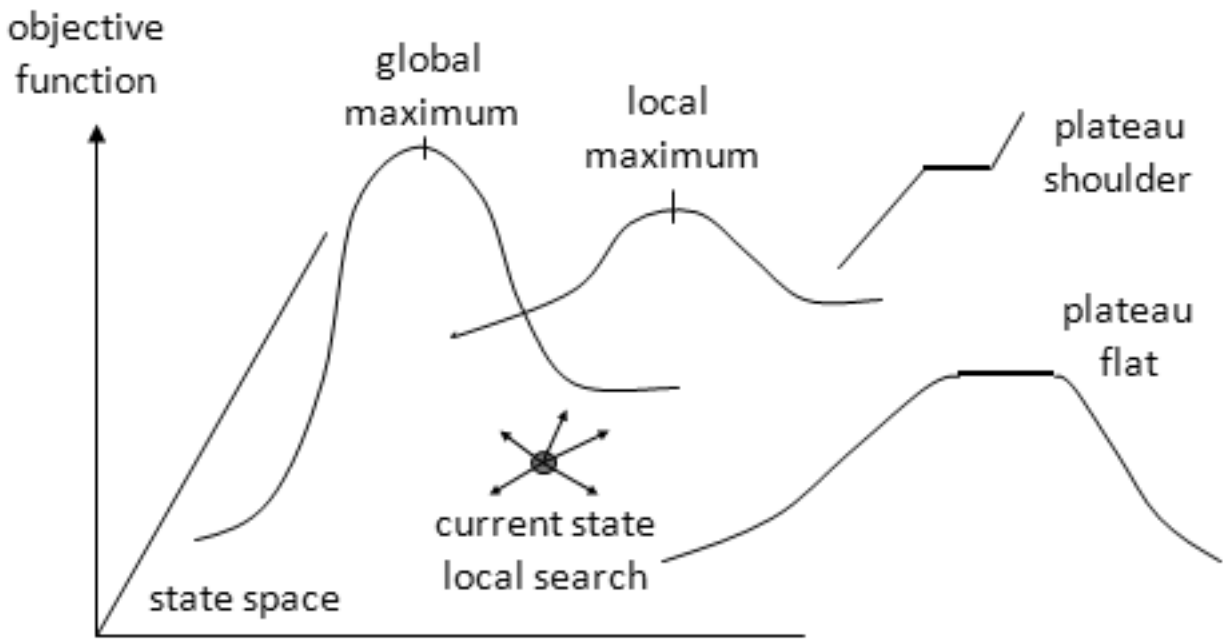


2. Hill climbing

Hill climbing is a general strategy, referring to a group of methods that share a common logic.

That is, the algorithm *generates neighbors* for the current state and *moves to a neighbor with a better objective function*, i.e., moves upwards in cases where maximizing the objective function is needed.

Hill climbing can be called *greedy-local search*.



2.1. Steepest-ascent hill climbing

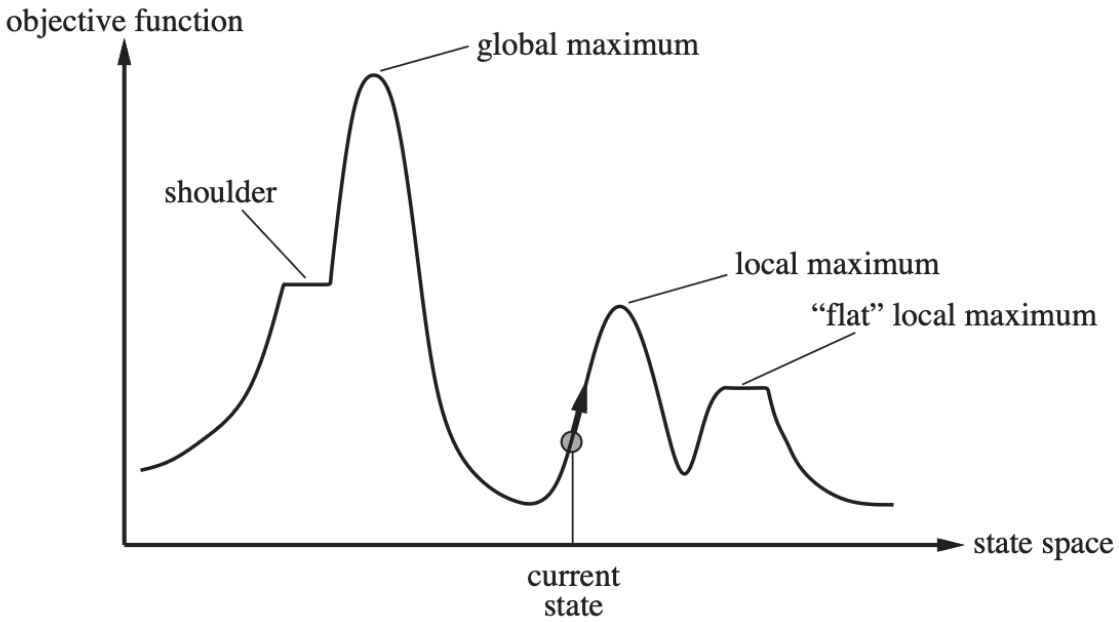
```
function HILL-CLIMBING(problem) returns a state that is a local maximum

    current ← MAKE-NODE(problem.INITIAL-STATE)
    loop do
        neighbor ← a highest-valued successor of current
        if neighbor.VALUE ≤ current.VALUE then return current.STATE
        current ← neighbor
```

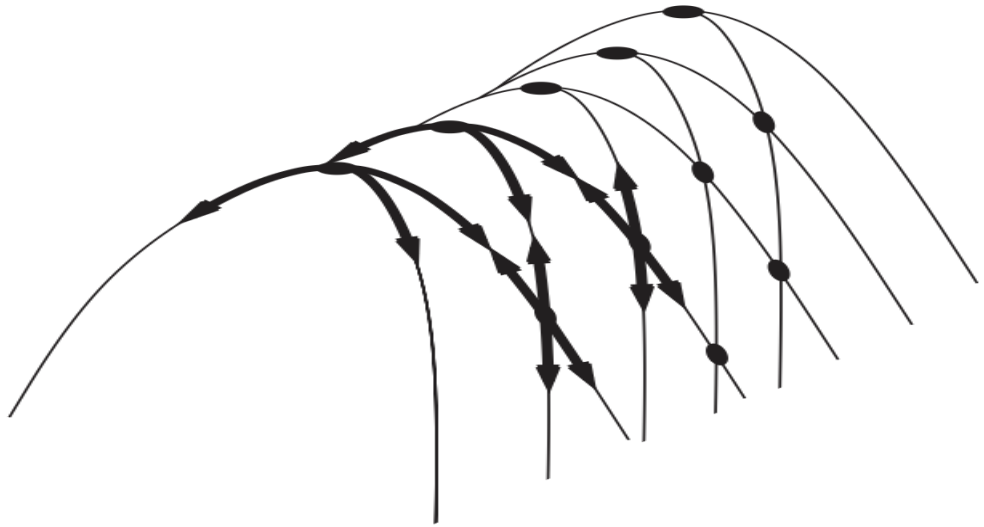
- Does not maintain a search tree.
- Does not look ahead beyond the immediate neighbors of the current state.

Limitations

Hill climbing can get stuck for the following reasons:

- Local maximum.
- Ridges.
- Plateau.





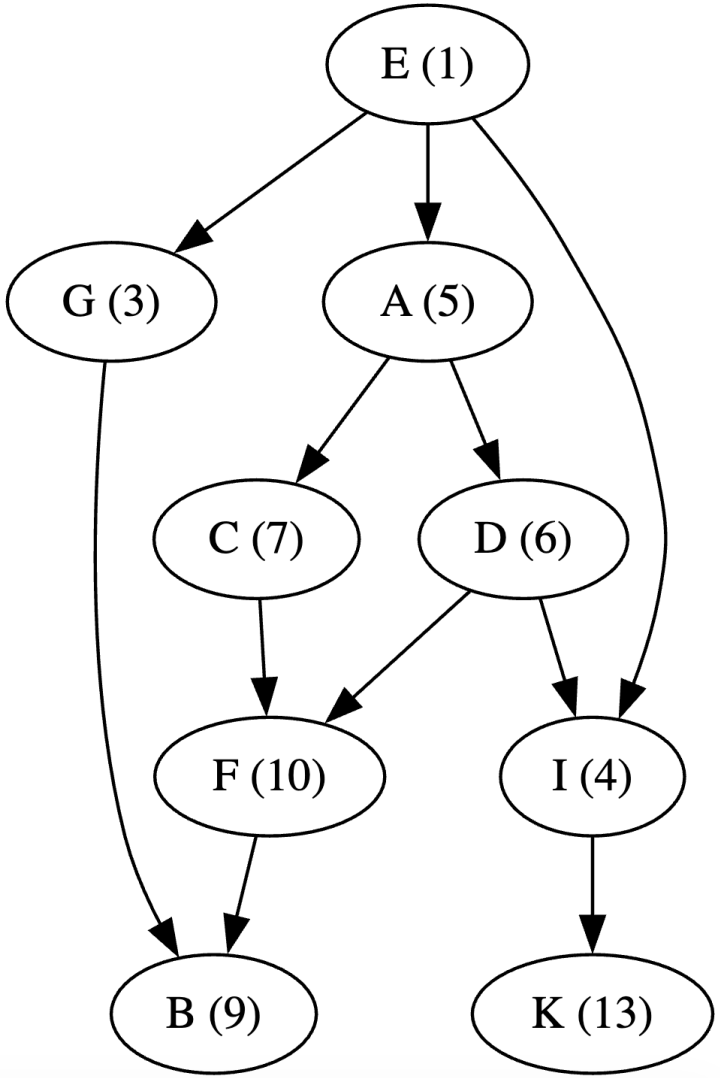
Example

Input: E

Procedure:

- Best = E
- Successor(E) = {A, G, I}
- f(A) > f(E) => Best = A
- Successor(A) = {C, D}
- f(C) > f(A) => Best = C
- Successor(C) = {F}
- f(F) > f(C) => Best = F
- Successor(F) = {B}
- f(B) < f(F) => Best = F.

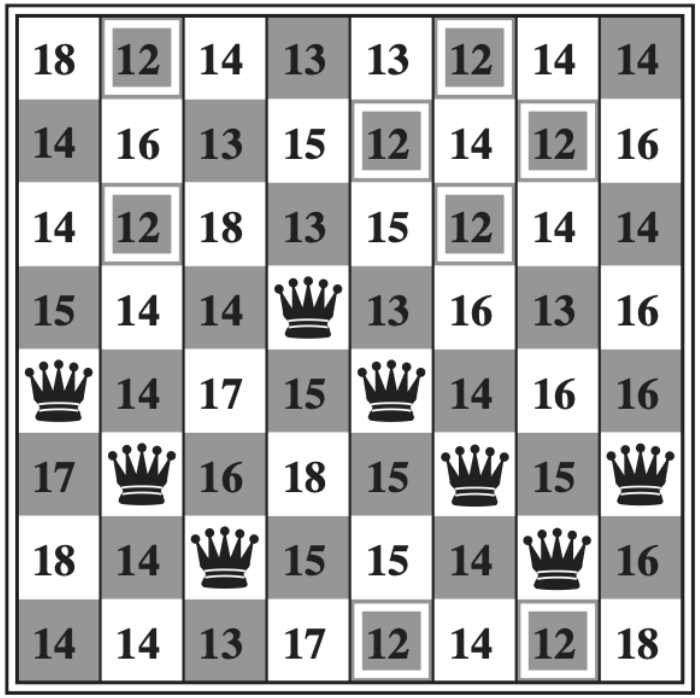
Return: F

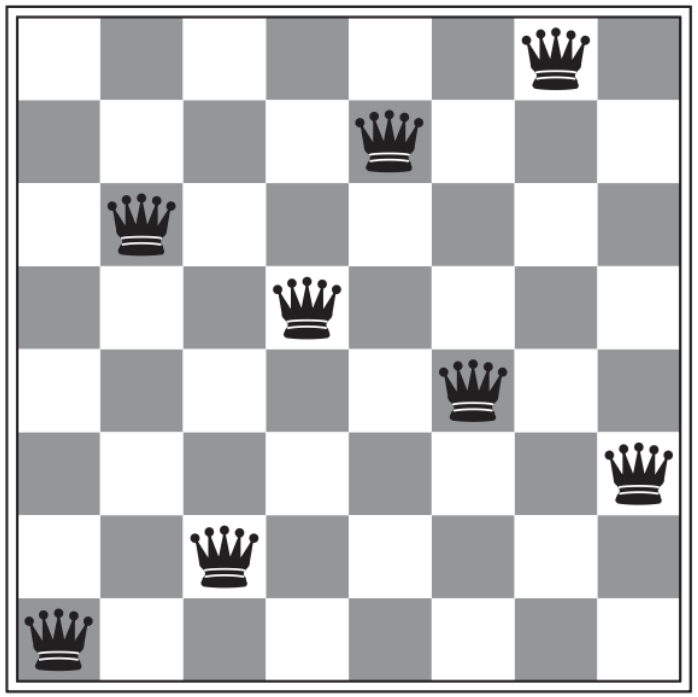


Importance of Successor

Successor design is crucial in steepest-ascent hill climbing:

- Too many neighbors → high computation cost
- Too few neighbors → may get stuck at local maxima





How many states will we have?

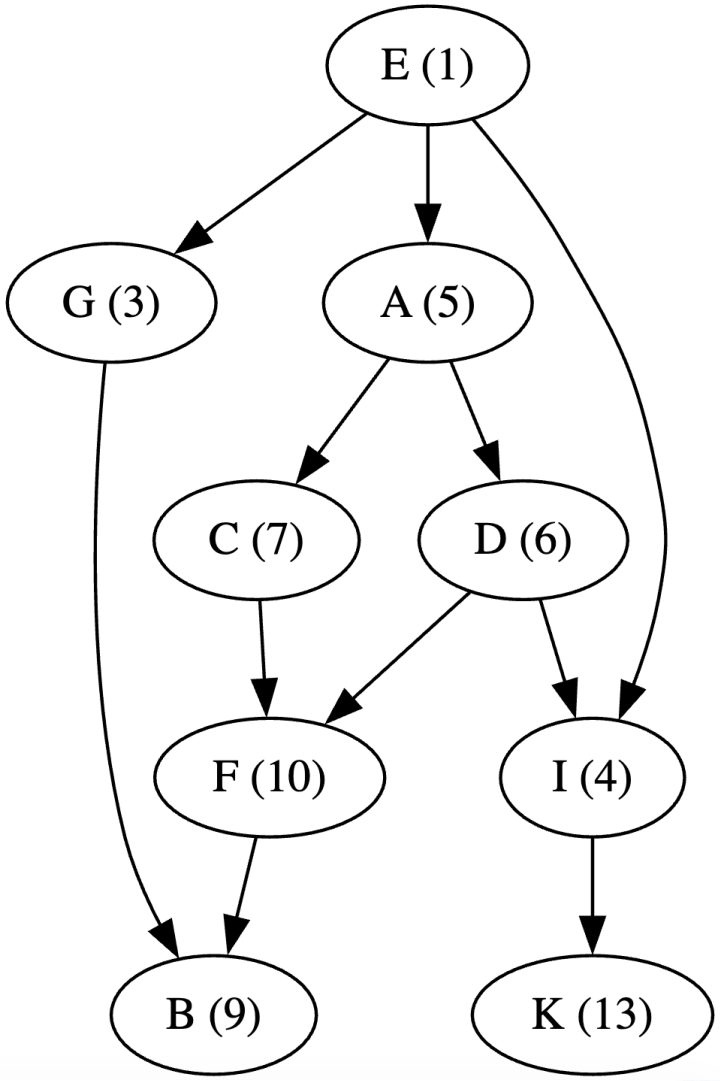
- Choose 1 column and move the queen.
- Choose 2 columns and move the queens.

2.2. Stochastic hill climbing

- Steepest-ascent hill climbing: the best is chosen; it can get stuck at a local maxima.
- Stochastic hill climbing: better is enough!

In the graph shown, if we randomly choose the better state, we may reach the global maximum after 2 steps.

E → I → K



2.3. Random restart hill climbing

1 Pick a random starting state
Randomly select a state from the state space.

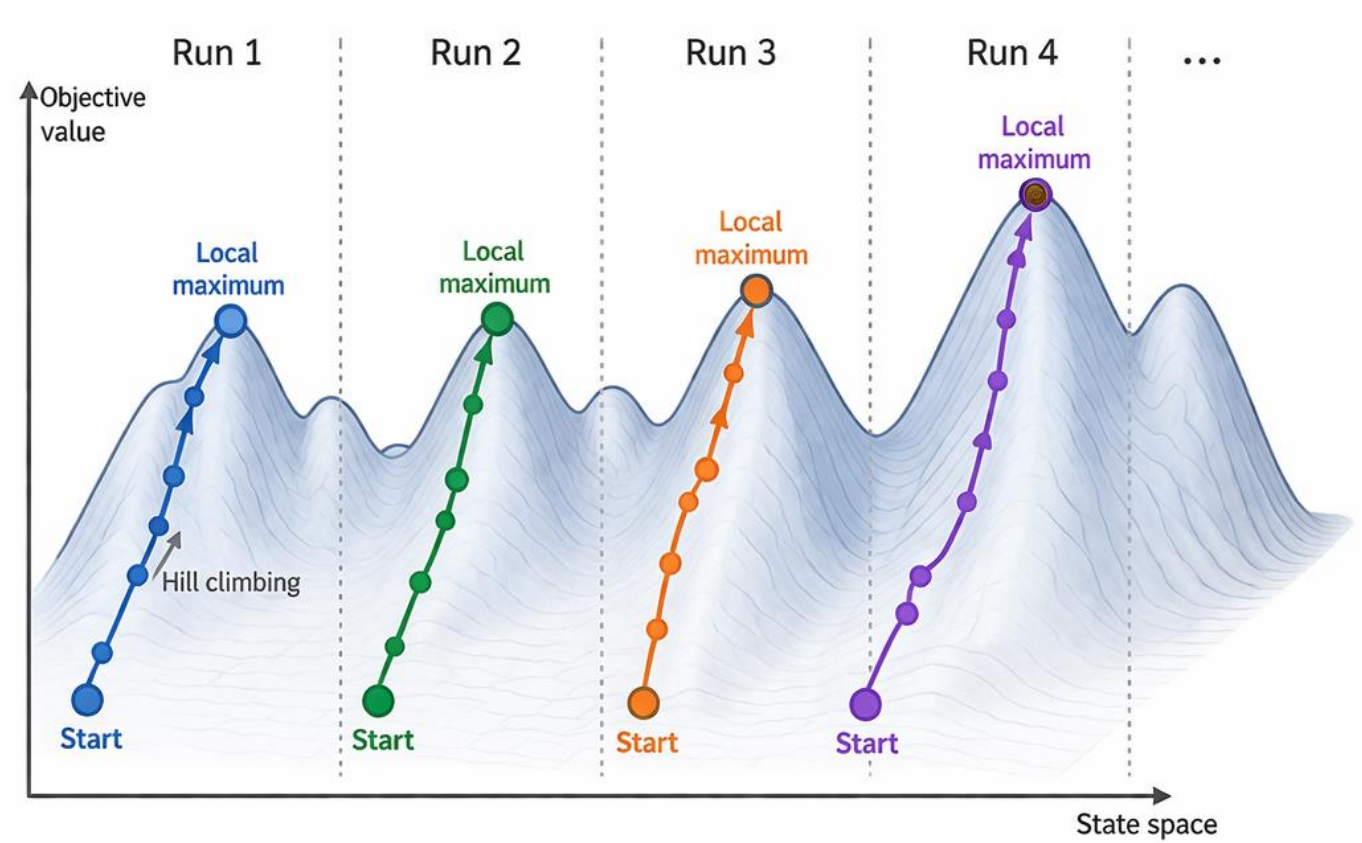
2 Run hill climbing
Move to the best neighbor as long as it improves the objective.

3 Store the result
Record the solution (local maxinum) found.

4 Repeat
Go back to Step 1 with a new random starting state.

5 End
Return the best solution among all runs.

Local maximum is still a problem!!!
Idea: Try and try again with different starting states.



2.4. Beam search

1 KEEP *k* STATES

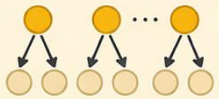
Keep track of *k* states rather than just one.

2 START

Start with *k* randomly generated states.

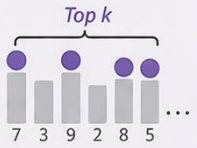
3 EXPAND

At each iteration, all the successors of all *k* states are generated.



4 SELECT TOP *k*

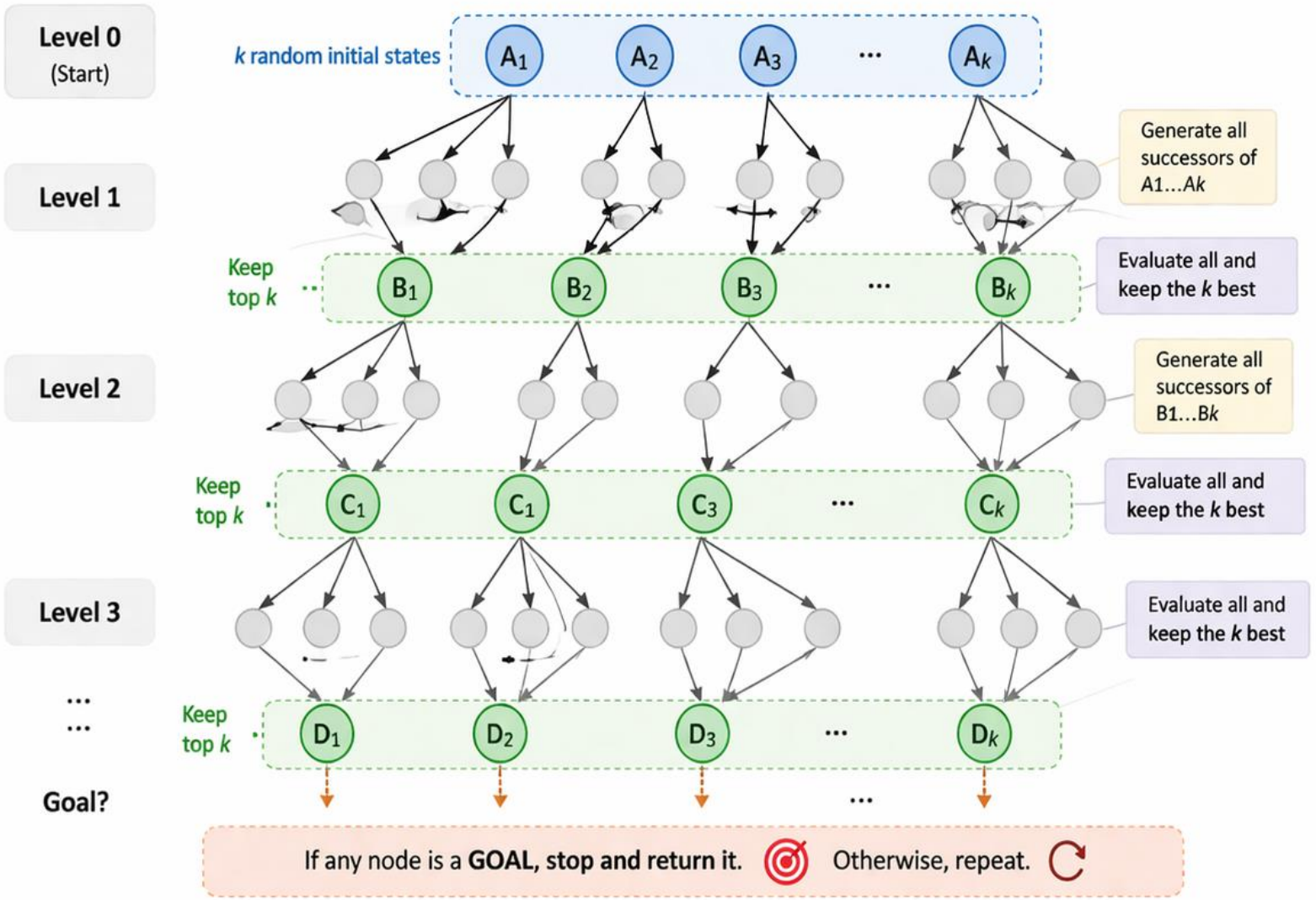
Select the *k* best successors from the complete list (by evaluation).



5 REPEAT / STOP

If any one is a goal state, stop; else repeat.

or



LEGEND

- A_i Current beam (size *k*)
- Generated successors
- B_i Next beam (top *k* states)
- Goal state
- Repeat

Intuition

Beam search explores multiple promising paths in parallel, but only keeps the best *k* at each level.

3. Simulated annealing

Idea: Escape local maxima by allowing **downhill moves**, but make them **rarer** as time goes on.

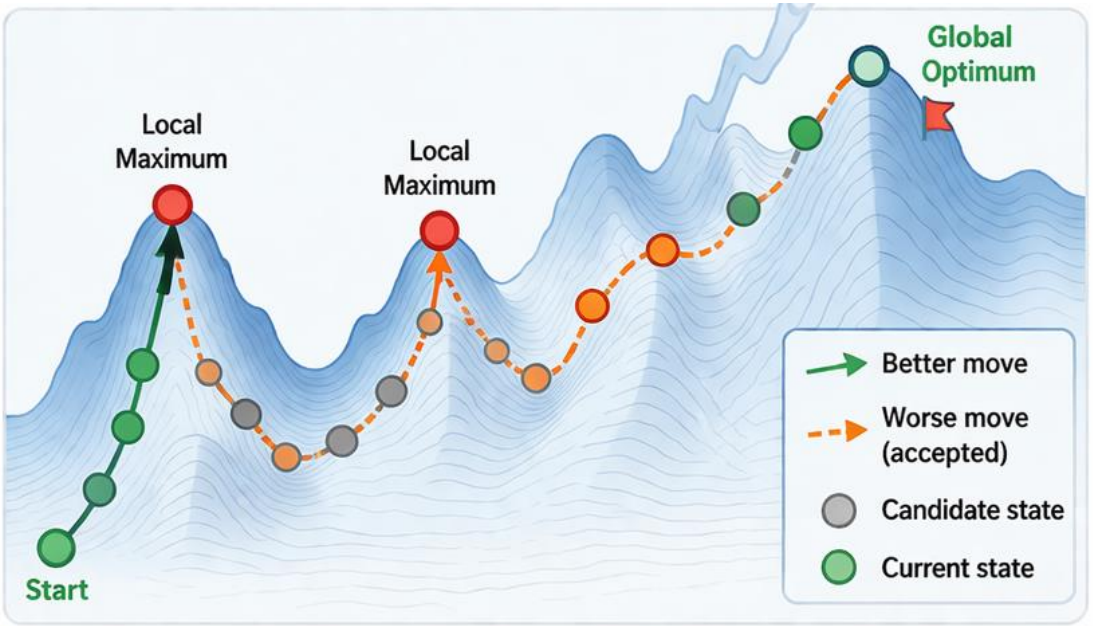
We are at state u. Choose a random neighbor **v**.

- If **v** is better than **u** ($cost(v) < cost(u)$): move to **v**.
- If **v** is not better than **u** ($cost(v) \geq cost(u)$): move to **v** with probability $e^{\Delta \frac{\Delta}{T}}$

where $\Delta = cost(v) - cost(u)$

‡ The probability decreases as a function of the "badness" of **v** and the temperature **T**. As **T** increases, the chance of moving to a worse state decreases.

Guaranteed probability: $e^{\Delta \frac{\Delta}{T}}$



Key intuition:

Early on (high **T**), we accept worse moves more often to explore and escape local maxima. Later (low **T**), we become more selective and focus on improving.

High T (explore) → Low T (exploit)

Acceptance probability: $e^{\frac{\Delta}{T}}$ where $\Delta = cost(v) - cost(u)$

3. Simulated annealing

Idea: Escape local maxima by allowing **downhill moves**, but make them **rarer** as time goes on.

function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state
  inputs: *problem*, a problem
    schedule, a mapping from time to "temperature"

  current ← MAKE-NODE(*problem*.INITIAL-STATE)
  for $t = 1$ **to** ∞ **do**
    $T \leftarrow schedule(t)$
    if $T = 0$ **then return** *current*
    next ← a randomly selected successor of *current*
    $\Delta E \leftarrow next.\text{VALUE} - current.\text{VALUE}$
    if $\Delta E > 0$ **then** *current* ← *next*
    else *current* ← *next* only with probability $e^{\Delta E/T}$

3. Simulated annealing

Idea

Sometimes accept **downhill moves**.... but become **less likely** over time.

Cooling

Temperature *T* decreases over time.

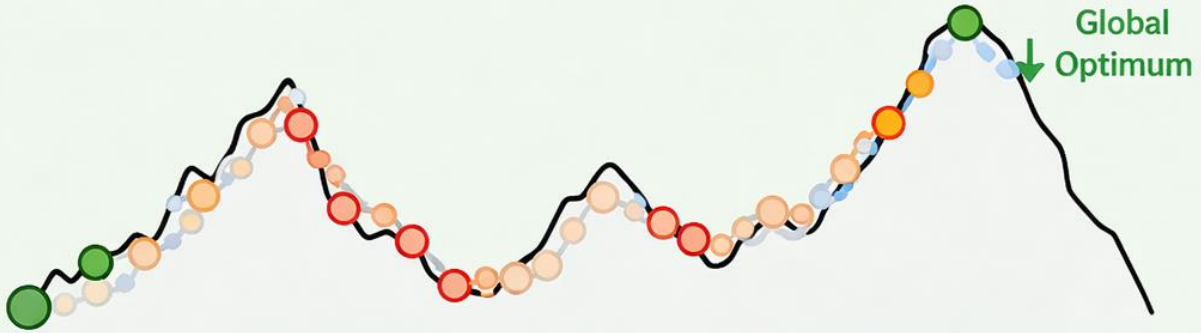
Accept worse moves more often

Accept worse moves sometimes

Accept worse moves rarely

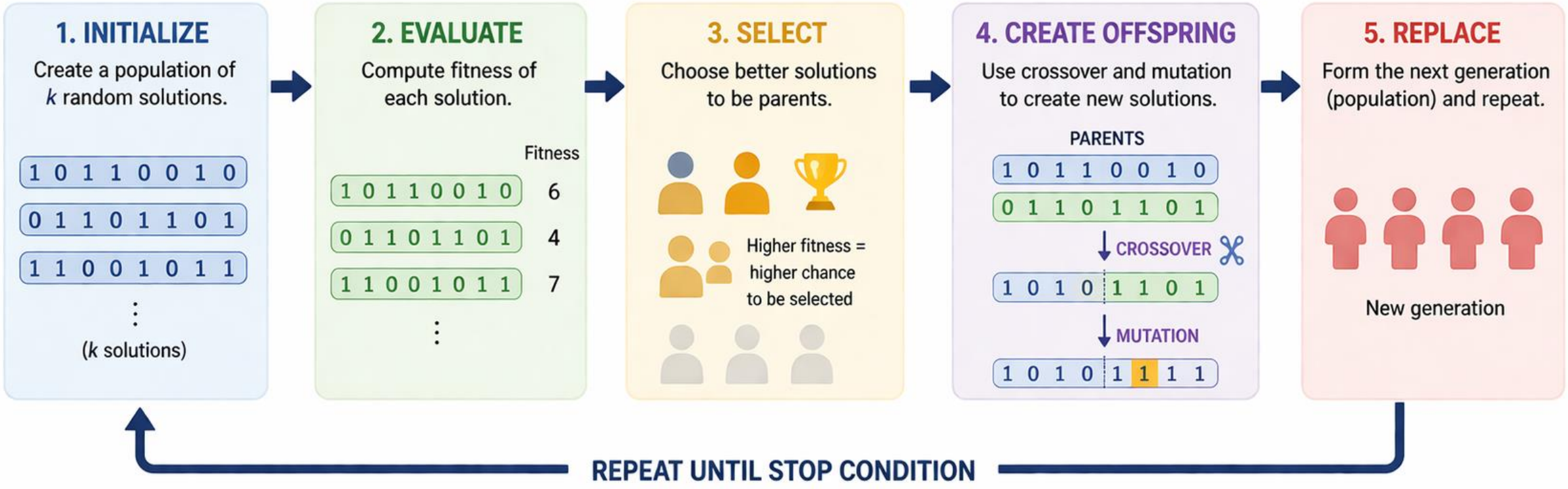
Result

If cooled slowly enough, it can find the global optimum.



4. Genetic algorithms

GA is an optimization method inspired by **natural evolution**.
It evolves a **population** of solutions to find better and better ones over time.



Goal: find the best solution (highest fitness).

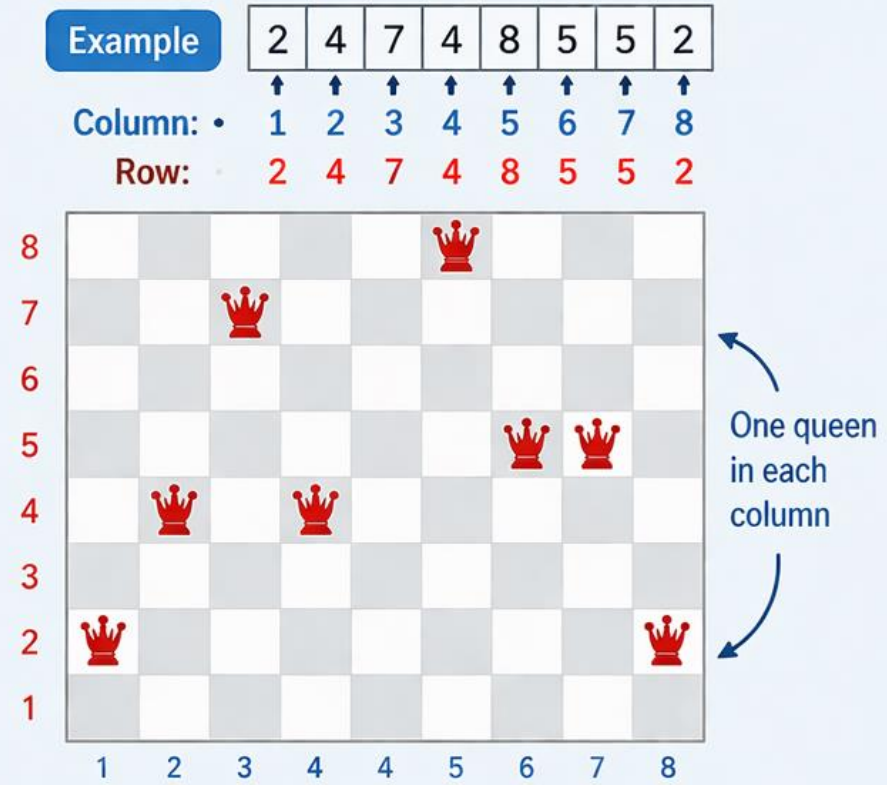
```
function GENETIC-ALGORITHM(population, FITNESS-FN) returns an individual
  inputs: population, a set of individuals
          FITNESS-FN, a function that measures the fitness of an individual

  repeat
    new_population ← empty set
    for i = 1 to SIZE(population) do
      x ← RANDOM-SELECTION(population, FITNESS-FN)
      y ← RANDOM-SELECTION(population, FITNESS-FN)
      child ← REPRODUCE(x, y)
      if (small random probability) then child ← MUTATE(child)
      add child to new_population
    population ← new_population
  until some individual is fit enough, or enough time has elapsed
  return the best individual in population, according to FITNESS-FN
```

```
function REPRODUCE(x, y) returns an individual
  inputs: x, y, parent individuals

  n ← LENGTH(x); c ← random number from 1 to n
  return APPEND(SUBSTRING(x, 1, c), SUBSTRING(y, c + 1, n))
```

Example

1. STATE REPRESENTATION

A state is represented as a string of 8 numbers.
The i-th number = row position of the queen in column i.



This encoding guarantees no two queens share the same column.

2. FITNESS FUNCTION

Fitness = number of pairs of queens that do NOT attack each other (no same row or diagonal).

Examples

State (string)	Board (queens)	Fitness
2 4 7 4 8 5 5 2		24
3 2 7 5 2 4 1 1		23
2 4 4 1 5 1 2 4		20
3 2 5 4 3 2 1 3		11

Max fitness: 8 × 7 / 2 = 28 (all pairs are non-attacking)

How it works (for the first example):

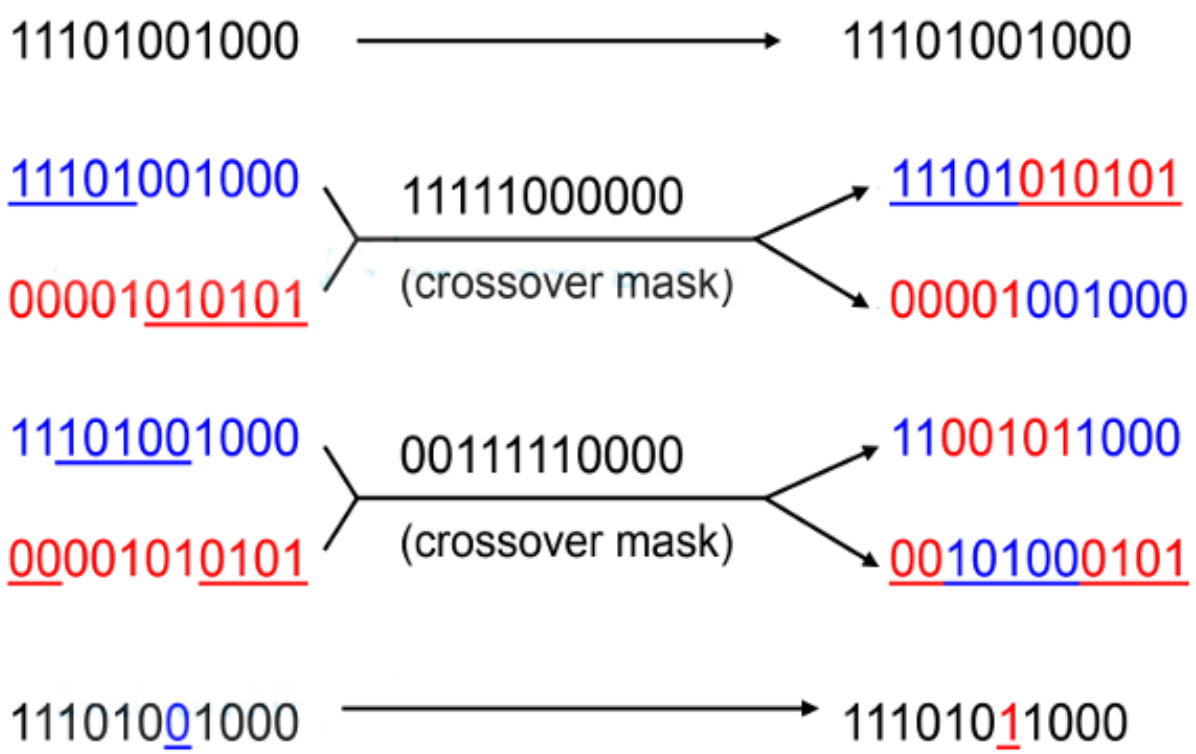
Higher fitness means a better state.

3 OPERATORS

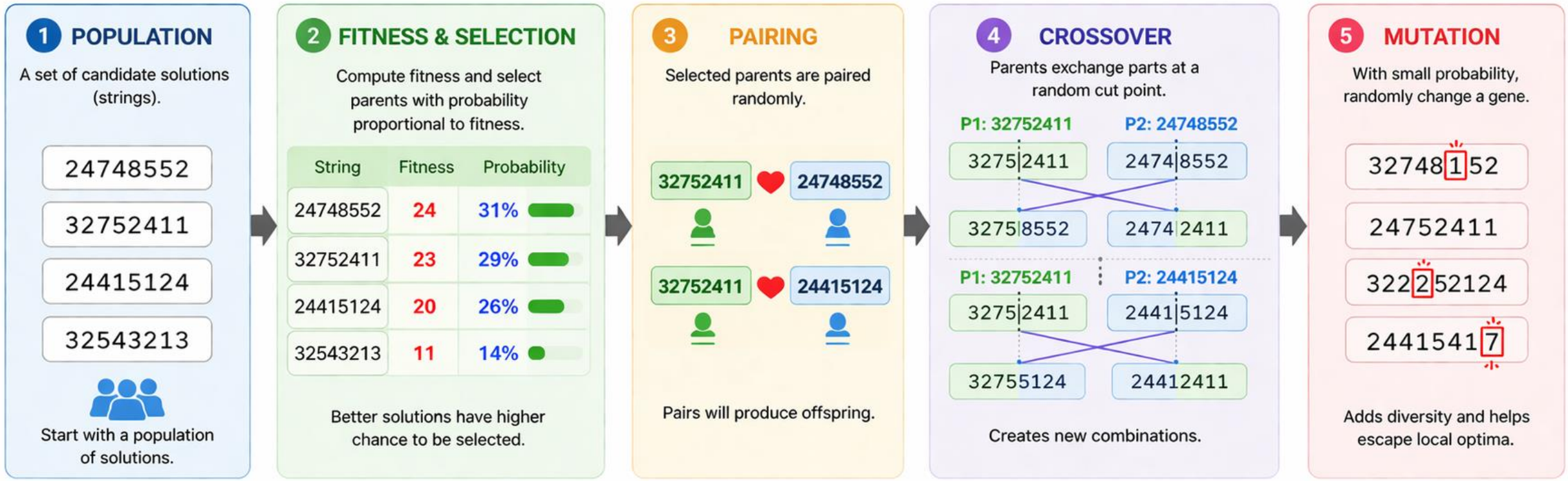
REPRODUCTION
(Copy)
Create offspring from selected parents.

CROSSOVER
Combine parts of two parents to create children.

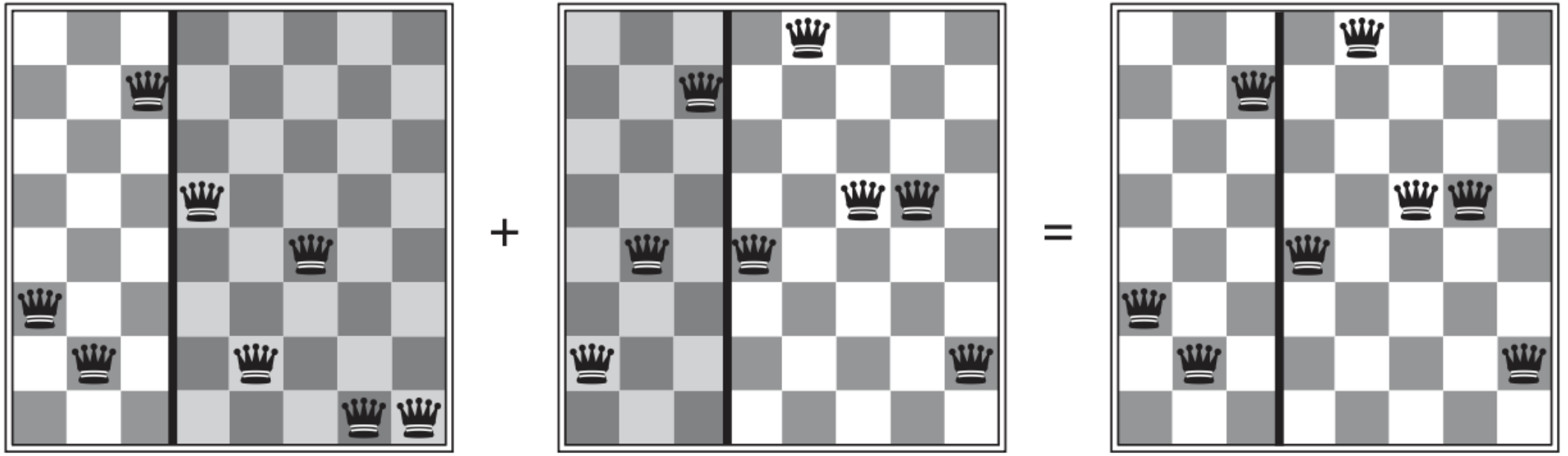
MUTATION
Randomly change a gene to maintain diversity.



How genetic algorithm works



In short: good solutions are more likely to survive, combine, and occasionally mutate. Over generations, the population improves.



32752411 + 24748552 = 32748552

Trung tâm Học liệu và Dạy học số

Đại học Công nghệ Kỹ thuật Tp. Hồ Chí Minh

01 Võ Văn Ngân, P. Thủ Đức, Tp.Hồ Chí Minh

daotaotuxa@hcmute.edu.vn